



МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Новосибирский государственный технический университет»



**НГТУ
НЭТИ**

**Факультет автоматики
и вычислительной техники**

Кафедра вычислительной техники
Расчетно-графическая работа
по дисциплине «Технология разработки программного обеспечения»

Java. Шаблоны проектирования

Студент Чумаков Илья

Группа АТМ-25

Преподаватель Токарев Вадим Геннадьевич

Новосибирск, 2025

1. Задание

Реализовать классический шаблон проектирования MVC на языке Java. В качестве предметной области необходимо разработать модель физического объекта с параметрическими зависимостями. Представление должно отображать параметры модели через текстовые поля. Каждый параметр должен быть связан с отдельной подпиской на события изменения в модели через единый интерфейс слушателя. Для управления каждым изменяемым параметром должен быть создан отдельный контроллер, обрабатывающий команды изменения значений. Результатом работы является работоспособное приложение, демонстрирующее принципы разделения ответственности между компонентами MVC и корректную работу событийной модели обновления представления.

2. Теоретическая часть

Шаблоны проектирования - это стандартизированные, проверенные на практике решения типовых проблем, возникающих при разработке программного обеспечения. Они представляют собой не готовый код, а архитектурные концепции, описывающие способы организации взаимодействия между компонентами системы. Основная цель применения шаблонов - повышение повторного использования кода, улучшение сопровождаемости, снижение связности между модулями и упрощение процесса проектирования сложных систем.

Одним из наиболее известных и широко применяемых архитектурных шаблонов является MVC (Model-View-Controller) - «Модель-Представление-Контроллер». Этот паттерн предлагает разделение приложения на три логически независимых компонента, каждый из которых отвечает за определённый аспект функциональности:

1. Модель (Model) — содержит данные предметной области и бизнес-логику. Модель не зависит от способа отображения данных или управления ими;

она лишь предоставляет методы для изменения своего состояния и уведомляет подписчиков об изменениях.

2. Представление (View) - отвечает за визуализацию данных модели. Представление подписывается на события изменения модели и обновляется автоматически при получении уведомлений. Оно не содержит бизнес-логики и не выполняет вычислений.

3. Контроллер (Controller) - обрабатывает пользовательский ввод, интерпретирует команды и инициирует изменения в модели. Контроллер выступает посредником между представлением и моделью, обеспечивая соблюдение бизнес-правил.

Ключевым принципом классического MVC является однонаправленность зависимостей:

- Контроллер знает о модели;
- Представление знает о модели (через механизм подписки);
- Модель не знает ни о представлении, ни о контроллере.

Такое разделение позволяет достичь слабой связности компонентов: модель может разрабатываться и тестироваться независимо от интерфейса, представление можно изменять или заменять без переписывания бизнес-логики, а контроллеры могут обрабатывать различные сценарии взаимодействия, не затрагивая внутреннюю структуру модели.

Преимущества использования MVC:

- Чёткое разделение ответственности между компонентами;
- Упрощение тестирования (модель и контроллер можно тестировать изолированно);
- Гибкость и удобство расширения системы;
- Возможность повторного использования модели в разных контекстах (например, в веб- и десктоп-приложениях).

Для реализации механизма подписки и уведомлений между компонентами MVC часто используется паттерн Observer (Наблюдатель). В такой связке модель выступает в роли издателя, который уведомляет подписанные представления об изменениях своего состояния, а представление может генерировать события пользовательского ввода, на которые подписан контроллер. Это классическое сочетание паттернов обеспечивает эффективную событийно-ориентированную коммуникацию между компонентами при сохранении слабой связанности архитектуры.

3. Описание разработки

В рамках работы была реализована система, демонстрирующая применение шаблона проектирования MVC для моделирования пружинного маятника (гармонического осциллятора). Система состоит из трёх основных компонентов:

3.1. Модель

Модель представляет собой физическую систему пружинного маятника, описываемую следующими параметрами:

Входные параметры:

- Масса груза m (кг);
- Жесткость пружины k (Н/м);
- Амплитуда колебаний (начальное отклонение) A (м);
- Время t (с)

Выходные параметры:

- Угловая частота $\omega = \sqrt{\frac{k}{m}}$ (рад/с)
- Период колебаний $T = \frac{2\pi}{\omega} = 2\pi \sqrt{\frac{m}{k}}$ (с)
- Частота $f = \frac{\omega}{2\pi} = \frac{1}{2\pi} \sqrt{\frac{k}{m}}$ (Гц)
- Положение(координата) в момент времени t $x(t) = A \cos(\omega t)$ (м)

- Скорость в момент времени t $v(t) = \frac{dx}{dt} = -A\omega \sin(\omega t)$ (м/с)
- Ускорение в момент времени t $a(t) = \frac{d^2x}{dt^2} = -A\omega^2 \cos(\omega t)$ (м/с²)

Модель реализует событийный механизм. При изменении любого входного параметра происходит пересчёт всех зависимых величин и уведомление подписчиков через интерфейс `ParameterListener`.

3.2. Представление (View)

Представление реализовано в виде консольного интерфейса, который отображает текущее состояние модели. Для каждого параметра представление подписывается на события его изменения и выводит обновлённые значения. Представление не содержит логики вычислений и полностью отделено от модели.

3.3. Контроллеры (Controller)

Для каждого изменяемого входного параметра реализован отдельный контроллер:

- `MassController`
- `StiffnessController`
- `AmplitudeController`
- `TimeController`

Каждый контроллер принимает команды на изменение значения параметра и передаёт их модели, обеспечивая принцип единственной ответственности и строгую декомпозицию логики управления.

4. Результаты тестирования

В ходе тестирования системы были проведены последовательные изменения входных параметров модели, чтобы убедиться в корректности работы механизма MVC, событийной подписки и пересчёта зависимых величин.

4.1. Начальное состояние системы

После запуска программы модель инициализируется значениями по умолчанию:

Параметр	Значение	Единица
Входные параметры		
Масса m	1.0000	кг
Жёсткость k	100.0000	Н/м
Амплитуда A	0.1000	м
Время t	0.0000	с
Вычисляемые параметры		
Угловая частота ω	10.0000	рад/с
Положение $x(t)$	0.1000	м
Скорость $v(t)$	0.0000	м/с
Ускорение $a(t)$	-10.0000	м/с ²
Период T	0.6283	с
Частота f	1.5915	Гц

Проверка вычислений:

$$\omega = \sqrt{\frac{k}{m}} = \sqrt{\frac{100}{1}} = 10.0 \text{ рад/с}$$

$$x(t) = A \cos(\omega t) = 0.1 \cos(0) = 0.1 \text{ м}$$

$$v(t) = -A\omega \sin(\omega t) = -0.1 \cdot 10 \cdot \sin(0) = 0.0 \text{ м/с}$$

$$a(t) = -A\omega^2 \cos(\omega t) = -0.1 \cdot 10^2 \cdot \cos(0) = -10.0 \text{ м/с}^2$$

$$T = \frac{2\pi}{\omega} = \frac{2\pi}{10} \approx 0.6283 \text{ с}$$

$$f = \frac{1}{T} = \frac{1}{0.6283} \approx 1.5915 \text{ Гц}$$

Все верно.

4.2. Изменение амплитуды

Установим амплитуду $A = 0.2$ м. Должны измениться положение скорость и ускорение. Но поскольку момент времени остался равен нулю, то скорость измениться не должна.

Параметр	Новое значение
Амплитуда (A)	0.2000 м
Положение($x(t)$)	0.2000 м
Скорость($v(t)$)	0.0000 м/с
Ускорение($a(t)$)	-20.0000 м/с ²

Так и произошло. Проверим вычисления:

$$x(t) = A \cos(\omega t) = 0.2 \cos(0) = 0.2 \text{ м}$$

$$a(t) = -A\omega^2 \cos(\omega t) = -0.1 \cdot 10^2 \cdot \cos(0) = -20.0 \text{ м/с}^2$$

4.3. Увеличение жёсткости пружины

Установим коэффициент жёсткости ($k = 200.0 \text{ Н/м}$). Должна измениться угловая частота, а вместе все остальные выходные параметры. Но т.к время все ещё равно 0, то изменения должны произойти только у периода, частоты и ускорения.

Параметр	Новое значение
Жёсткость (k)	200.0000 Н/м
Угловая частота (ω)	14.1421 рад/с
Период (T)	0.4443 с
Частота (f)	2.2508 Гц
Ускорение ($a(t)$)	-40.0000 м/с ²

Именно эти характеристики и изменились. Проверим правильность вычислений.

$$\omega = \sqrt{\frac{200}{1}} \approx 14.1421 \text{ рад/с}$$

$$T = \frac{2\pi}{14.1421} \approx 0.4443 \text{ с}$$

$$f = \frac{1}{0.4443} \approx 2.2508 \text{ Гц}$$

$$a(0) = -0.2 \cdot (14.1421)^2 \cdot \cos(0) \approx -40.0 \text{ м/с}^2$$

4.4. Уменьшение массы груза

Установим массу груза ($m = 0.5 \text{ кг}$). Вновь должна измениться угловая скорость. И те же характеристики, что и в предыдущем тесте.

Параметр	Новое значение
Масса (m)	0.5000 кг
Угловая частота (ω)	20.0000 рад/с
Период (T)	0.3142 с
Частота (f)	3.1831 Гц
Ускорение($a(t)$)	-80.0000 м/с ²

Вновь изменились только нужные параметры. Поскольку другие параметры используют угловую для их расчета проверим в этот раз только её:

$$\omega = \sqrt{\frac{200}{0.5}} = 20.0 \text{ рад/с}$$

4.5. Изменение времени

Установим время ($t = 0.1571 \text{ с}$)(примерно($T/2$)). Должны измениться «скоростные» характеристики.

Параметр	Новое значение
Время (t)	0.1571 с
Положение($x(t)$)	-0.2000 м
Скорость($v(t)$)	- 0.0016 м/с
Ускорение($a(t)$)	80.0000 м/с ²

Проверим вычисления:

$$\omega t = 20 \cdot 0.1571 = 3.142 \approx \pi$$

$$x(t) = 0.2 \cos(\pi) = -0.2 \text{ м}$$

$$v(t) = -0.2 \cdot 20 \cdot \sin(\pi) = 0 \text{ м/с}$$

$$a(t) = -0.2 \cdot 20^2 \cdot \cos(\pi) = 80 \text{ м/с}^2$$

Расхождения в вычисленном значении скорости связаны с моим приближенным вычислением ωt .

5. Итоги

В ходе лабораторной работы был успешно реализован классический шаблон проектирования MVC на языке Java на примере модели пружинного маятника.

Основные результаты:

- Чёткое разделение ответственности. Модель, представление и контроллеры были строго разделены, что соответствует принципам MVC.
- Корректная реализация событийной модели. Система использует механизм подписки, что позволяет представлению автоматически обновляться при изменении модели без прямых зависимостей.
- Расширяемость архитектуры. Добавление новых параметров или контроллеров не требует изменения существующего кода.

- Физическая корректность модели. Все вычисления соответствуют формулам гармонического осциллятора, что подтверждено тестированием.

Система демонстрирует:

- Слабую связность компонентов
- Удобство поддержки и модификации
- Корректную работу в реальном времени при изменении параметров

Работа подтвердила эффективность использования шаблона MVC для построения модульных, гибких и поддерживаемых приложений.

6. Список литературы

1) Стелтинг С., Маассен О. Применение шаблонов Java. Библиотека профессионала : пер. с англ. — М. : Вильямс, 2002. — 576 с.

2) Часть 7. Знакомство с паттерном MVC (Model-View-Controller) [Электронный ресурс] // JavaRush. URL: <https://javarush.com/groups/posts/2536-chastjh-7-znakomstvo-s-patternom-mvc-model-view-controller>